S-notation

S-Notation Markup Rules Technical Description

Prepared by: Eric Van Horenbeeck, Tom Pauwaert,
Luuk Van Waes, Mariëlle Leijten

Version: 1.0

Version history: v 0.0 July 2012 - Antwerp meeting
v 1.0 July 2012 - Presentation Sig Writing conference in Porto

Date: July 7, 2012

Project Members: **Inputlog (Belgium)**
Luuk Van Waes - University of Antwerp
Mariëlle Leijten - Flanders Research Foundation/UA
Eric Van Horenbeeck - University of Antwerp
Tom Pauwaert - University of Antwerp

S-notation is a computer-based method for tracing the writing process. It is first fully described by Py Kollberg[1] from the Swedish Royal Institute of Technology in 1997 and is based on an informal notation for revisions in handwriting published 10 years earlier by Matsuhashi[2]. Insertions, deletions, and breaks are numbered according to the order of their occurrence in the writing process. It allows describing writing as a process. One can see at what point in the text a decision was made to delete an earlier production or to insert new characters or phrases and to see where these revisions occur. By convention a 'pipe' symbol (sometimes called 'vertical tab') is used to indicate a break. Curled braces indicate an insertion and square brackets surround a deletion. These symbols are sequentially numbered.

Terms used in this overview

symbol	anything printable
position	the exact location of a character in the text of a document. Certain symbols do not have an effect of on other symbols' location.
positionCount	0 for all markup symbols, 1 for all undeleted text characters. A deleted character gets a positionCount of 0. Passing a symbol with a positionCount of 0 does not have an effect on subsequent symbols' location.
markup	the symbols used to render the break, insertion or deletion
char	a text character (a space is also considered a character)
brace	{ *or* }
bracket	[*or*]

1. Normal Production

Definition: normal production is any production of characters at the end of the document. The end of the document is either behind the last position in the document or at a position in the document that only has whitespace trailing behind.
Normal production is not versioned and is not marked by any sort of markup.

2. Break

Defintion: a break is an interruption of the writing process. A break is placed at the last edit position within the text, before jumping to another location in the text to perform an insertion or deletion. When an insert or a delete is followed by a normal production, no break is inserted. A break symbol is followed by the revision number indicating the revision the break is associated with.

E.g.: a break is inserted at the end of the normal production 'ababa', before the insertion of 'cdf'.	abab{cdf}$_1$a|$_1$
Following normal production does not trigger a break at the end of the insertion of 'cdf'	abab{cdf}$_1$a|$_1$baba

[1] Kollberg, P. (1997). *S-Notation as a Tool for Analysing the Episodic Structure of Revisions*. Paper presented at the European Writing Conferences, Barcelona (ES)
[2] Matsuhashi, A.(1987). "Revising the plan and altering the text" In *Writing in real time -Modeling production processes*, ed. A. Matsuhashi. 197-223. Norwood, New Jersey:Ablex Publ.

3. Insertion

Definition: an insertion is the addition of characters inside the already produced text. Before the insertion is started a break symbol is inserted at the last edit position before the insertion of the characters.
Each insertion that is not normal production keeps track of the number of 'active characters' it contains. Active characters are non-markup characters with a positionCount > 0.

The break symbol has the same version number as the closing bracket of the insertion.

abab{cdf}$_1$a|$_1$baba

Insertions can be nested but the insertion markup but the insertion markup can never cross each other. Furthermore, only newer inserts can be nested in older ones.

abab{cd{e}$_2$f}$_1$a|$_1$baba|$_2$

When nesting an insertion within an older insertion, the older insertions active character incount will increase by an amount equal to the number of active characters in the newer, nested insertion. The insertions are from that point onward 'linked' into a chain of nested insertions. Any insertion within another insertion will ripple an increase in active character count through all the insertions it is nested in.
Likewise, any deletion of characters within a nested insertion will reduce the active character count of all insertions it is nested in.

insert^1 : 3 + 1 active characters
insert^2 : 1 active character

4. Deletion

Definition: the removal of one or more characters in the produced text.

The break symbol has the same version number as the closing bracket of the deletion.

[ab]$_3$ab{cd{e}$_2$f}$_1$a|$_1$baba|$_2$|$_3$

The rest of the properties of a deletion are defined by the jumping rules which are explained in the next section.

5. Constructing the S-Notation

The construction of the S-Notation is build upon two important parts. First the exact location for the insertion has to be determined, this will be described in the next section. Besides the location the jumping rules of the markup are determining for the S-Notation result string.

5.1 Finding the location

When finding the location in the S-Notation string under construction we start counting positions from the left hand side and count. Consider the following example string.

We have to find position 9 for the insertion of a the text 'foo'.

[ab]$_3$ab{cd{e}$_2$f}$_1$a|$_1$baba|$_2$|$_3$

Markup symbols have positionCount 0 and do not

[ab]$_3$ab{cd{e}$_2$f}$_1$a|$_1$baba|$_2$|$_3$

increase position	^ position = 0
Deleted symbols have positionCount 0 and do not increase the position	$[ab]_3ab\{cd\{e\}_2f\}_1a\|_1baba\|_2\|_3$ ^ position = 0
Markup symbols - positionCount 0	$[ab]_3ab\{cd\{e\}_2f\}_1a\|_1baba\|_2\|_3$ ^ position = 0
Text symbols that are not deleted: positionCount 1	$[ab]_3ab\{cd\{e\}_2f\}_1a\|_1baba\|_2\|_3$ ^ position = 2
Markup symbol - positionCount 0	$[ab]_3ab\{cd\{e\}_2f\}_1a\|_1baba\|_2\|_3$ ^ position = 2
This continues on until the correct position is found.	$[ab]_3ab\{cd\{e\}_2f\}_1a\|_1baba\|_2\|_3$ ^ position = 9

Note that if we have to insert the text 'foo' at position 9, this means we have to insert the text before the character 'a'. As positionCount only increases on actual text symbols, the insertion point will always be before an actual text character, or at the end of the document but never we will never end on a markup symbol or on a deleted symbol. It is also important to realise that there is a big difference between:
- Case 1: moving to position 8 and inserting after the 'b'
- Case 2: moving to position 9 and inserting before the 'a'

Consider the above example with an extra deletion in version 4, before the insertion of the text 'foo':

$[ab]_3|_4ab\{cd\{e\}_2f\}_1a|_1b[a]_4ba|_2|_3$
^

position = 9

Case 1:

Moving to position 8:	$[ab]_3\|_4ab\{cd\{e\}_2f\}_1a\|_1b[a]_4ba\|_2\|_3$ ^ position = 8
Inserting the text 'foo' would lead to lead to 'foo' being inserted before the delete of the 'a' in version 4.	$[ab]_3\|_4ab\{cd\{e\}_2f\}_1a\|_1b\{foo\}_5[a]_4\|_5ba\|_2\|_3$ ^ position = 8

Case 2:

Moving to position 9:	$[ab]_3\|_4ab\{cd\{e\}_2f\}_1a\|_1b[a]_4ba\|_2\|_3$ ^ position = 9
Inserting the text 'foo' before the 'b' would make sure that we are passed any removed characters or skippable markup and provides more readability	$[ab]_3\|_4ab\{cd\{e\}_2f\}_1a\|_1b[a]_4\|_5\{foo\}_5ba\|_2\|_3$ ^ position = 12

For this reason Inputlog moves past the location and then inserts before the insertion point as in case 2.

5.2 Markup Jumping Rules

Certain markup symbols may jump over other markup symbols according to the jumping rules. This is mostly for readability reasons. The jumping rules depend on the symbol attempting to move itself, the symbols it tries to jump past, and the revision numbers of both symbols.
The markup symbols location is first determined, afterwards we check if
- A break can pass over other markup symbols present to its direct right.
- An 'open' markuptag (left bracket [or left brace {) can pass over any markup symbols to its direct left.
- A 'close' markuptag (right bracket $]_x$ or right brace $\}_x$) can pass over any markup symbols to its direct right.

Rules:
- A **break symbol** $|_x$ may only jump over another break symbol if it has a higher revision number than the break it is attempting to pass.
- An **open insert markup symbol '{'** may not jump over any other symbols.
- A **close insert markup symbol '$\}_x$'** may not jump over any other symbols.

Delete markup has more interesting jumping rules. We keep track of the number of characters to be removed in the delete revision. First the start location of the deletion is determined. From there we determine the location of the **delete close markup symbol '$]_x$'** by moving the closing tag to the right as long we have characters to be removed or markup can be jumped. Everytime a character is passed that has a positionCount = 1, its positionCount is set equal to 0, considered deleted, and the number of characters that still have to be deleted is decremented by 1.

The **delete close markup symbol '$]_x$'** jumps when there are **still characters to be deleted:**
- Over any other symbol.

The **delete close markup symbol '$]_x$'** jumps when there are **no more characters left to be deleted** over:
- breaks symbols of a revision strictly older than the delete revision.
- other delete tags
- any non markup node with a positionCount equal to 0, thus any previously deleted character.
- any insertion open or close tag IF that insertion has no active characters left.

The **delete open markup symbol '['** is only inserted in the S-Notation string after the closing symbol has already been placed and consequentially, no more characters still have to be deleted. Any insertion markup symbols at this point have an updated number of active characters remaining after this specific delete revision. The delete symbol jumps over:
- all symbols with positionCount equal to 0, except insertion markup symbols if that respective insert revision has an active character count > 0. If the insertion revision of those insertion markup symbols has no more active characters, those insertion markup symbols may also be jumped.

6. Extended Example

Consider following example:
We produce the following string in normal production:
"these is prodtion"

Revision 1: Insert 'normal ':
"these is **{normal }**$_1$prodtion**|**$_1$"
Revision 2 & 3: Change the word 'these' to 'this':
"th**[ese]**$_2$ is {normal }$_1$**|**$_2$prodtion|$_1$"
"th[ese]$_2$**|**$_3$**{is}**$_3$ is {normal }$_1$|$_2$prodtion|$_1$"

Revision 4: Correct the word 'prodtion':
"th[ese]$_2$|$_3${is}$_3$**|**$_4$ is {normal }$_1$|$_2$prod**{uc}**$_4$tion|$_1$"

Continue producing text (**normal production -> no revision**):
"th[ese]$_2$|$_3${is}$_3$|$_4$ is {normal }$_1$|$_2$prod{uc}$_4$tion|$_1$ **people**"

Revision 5: Insert the word 'not':
"th[ese]$_2$|$_3${is}$_3$|$_4$ is {**{not }**$_5$normal }$_1$|$_2$prod{uc}$_4$tion|$_1$ people**|**$_5$"

Note:
Insert Revision 5 has active character count 4.
Insert Revision 1 has active character count 7 + 4 = 11.

Revision 6: Remove 'ot ':
"th[ese]$_2$|$_3${is}$_3$|$_4$ is {{n**[ot]**$_6$}$_5$**|**$_6$normal }$_1$|$_2$prod{uc}$_4$tion|$_1$ people|$_5$"

Insert Revision 5 has active character count 1.
Insert Revision 1 has active character count 7 + 1 = 8.

Revision 7: Remove 'nnormal ':
"th[ese]$_2$|$_3${is}$_3$|$_4$ is {{n[ot]$_6$**|**$_7$}$_5$|$_6$normal }$_1$|$_2$prod{uc}$_4$tion|$_1$ people|$_5$"
^
We start at the postion before the first n with 8 characters to be deleted.
- We pass the 'n' > 7 characters to be deleted.
- We pass [ot]$_6$ while still having more than 0 characters to be deleted > jump
- We pass }$_5$ while still having more than 0 characters to be deleted > jump
- We pass |$_6$ a break symbol with revision number equal to our own revision number, but as we still have more than 0 characters to be deleted > jump
- We pass 'normal '. At this point we have 0 characters left to be deleted.

At this point:
Insert Revision 5 has active character count 0.
Insert Revision 1 has active character count 0 + 0 = 0.

- We pass }$_1$ with 0 characters left to be deleted. But as revision 1 has 0 active characters our delete close tag still jumps over the insertion close tag.
- We pass |$_2$ a break symbol with revision number lower than our own revision number, we jump.
- We pass 'p', a character with positionCount > 0. As we don't have any more characters to be deleted, we cannot pass 'p'. The delete close tag is inserted before before character 'p'.

"th[ese]$_2$|$_3${is}$_3$|$_4$ is {{**n[ot]$_6$|$_7$}$_5$|$_6$normal }$_1$|$_2$]$_7$prod{uc}$_4$tion|$_1$people|$_5$"
^

We want to insert the left deletion open tag '['. We start at 'n' and try to pass over symbols to the left.
- We pass the insertion opening bracket '{' of revision 5. As insert revision 5 has an active character count of 0 we jump over the symbol.
- We pass the insertion opening bracket '{' of revision 1. As insert revision 1 has an active character count of 0 we jump over the symbol.
- We pass character ' ', as ' ' has a positionCount > 0 we cannot pass it, we insert the deletion opening bracket behind the ' '.

"th[ese]$_2$|$_3${is}$_3$|$_4$ is **[{{n[ot]$_6$|$_7$}$_5$|$_6$normal }$_1$|$_2$]$_7$**prod{uc}$_4$tion|$_1$people|$_5$"